

Introduction to the Theory of Computation

Lecture 5. Regular Languages: Closure Properties and Nondeterminism

Neo (Yuan) Li

Sep 2026

Fudan University

yuan_li@fudan.edu.cn

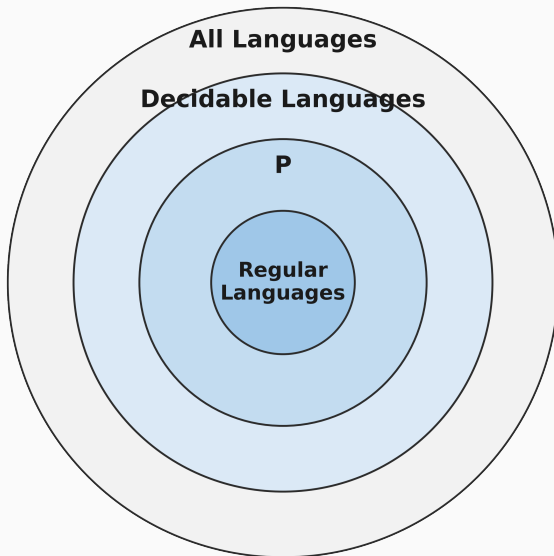
Definition: Regular Languages

Definition 1

A language is called a *regular language* if some finite automaton recognizes it.

If a machine accepts no strings, then it recognizes one language, namely the **empty language**.

The Language Hierarchy



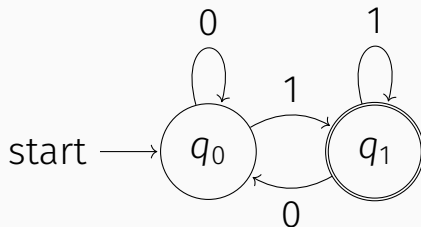
Example 1: Strings Ending with 1

Let language

$$L = \{ w \in \{0,1\}^* : w \text{ ends with } 1 \}.$$

Claim: L is a regular language.

The following DFA recognizes L .



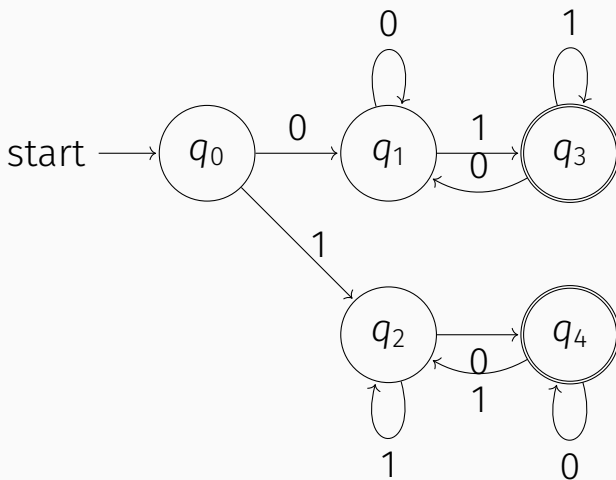
Example 2: First and Last Bit Differ

Let language

$$L = \{ w \in \{0,1\}^* : w = w_1w_2 \cdots w_n, \\ w_i \in \{0,1\}, n \geq 2, w_1 \neq w_n \}.$$

Claim: L is a regular language.

The following DFA recognizes L .



Exercise

For each language below, determine whether it is regular.

1. $L_1 = \{ w \in \{0, 1\}^* : w \text{ contains } 001 \text{ as a substring} \}$
2. $L_2 = \{ w \in \{0, 1\}^* :$
 $w \text{ does not contain } 001 \text{ as a substring} \}$
3. $L_3 = \{ 0^n 1^n : n \geq 0 \}$
4. $L_4 = \{ w \in \{0, 1\}^* :$
 $w \text{ has an equal number of } 0\text{'s and } 1\text{'s} \}$
5. $L_5 = \{ 0^n 1^m : n, m \geq 0 \}$

Closure Properties of Regular Languages

Let A and B be languages.

- (Complement) $\bar{A} = \{x \in \Sigma^* : x \notin A\}$
- (Union) $A \cup B = \{x : x \in A \text{ or } x \in B\}$
- (Intersection) $A \cap B = \{x : x \in A \text{ and } x \in B\}$
- (Concatenation)
 $A \circ B = \{xy : x \in A \text{ and } y \in B\}$
- (Star)
 $A^* = \{x_1x_2 \cdots x_k : k \geq 0 \text{ and each } x_i \in A\}$

Example

Let $A = \{0^i : i \geq 0\} = \{\epsilon, 0, 00, 000, \dots\}$ and $B = \{1^j : j \geq 0\} = \{\epsilon, 1, 11, 111, \dots\}$.

- $A \circ B = \{0^i 1^j : i, j \geq 0\}$
- $A^* = A$
- $B^* = B$
- $(A \circ B)^* = \{0, 1\}^*$

Let us show that regular languages are **closed** under

- Complement ($\bar{}$)
- Union ($\cup$)
- Intersection ($\cap$)
- Concatenation ($\circ$)
- Star ($*$)

Closure under Complement

Theorem 2

If L is a regular language, then its complement $\bar{L}$ is also regular.

Proof.

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA recognizing L . Let $M' = (Q, \Sigma, \delta, q_0, F')$, where

$$F' = Q \setminus F,$$

i.e., swap accept and reject states. Then M' accepts exactly the strings M rejects, so $L(M') = \bar{L}$. □

Theorem 3

If L_1, L_2 are regular languages, then $L_1 \cup L_2$ is also regular.

Proof idea: run M_1 and M_2 **simultaneously** using the **Cartesian product** of their states.

Union: The Cartesian Product Construction

Proof.

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize L_1 , and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize L_2 .

Construct $M = (Q, \Sigma, \delta, q_0, F)$ to recognize $L_1 \cup L_2$ as follows:

1. $Q = Q_1 \times Q_2$
2. Σ is the same alphabet as in M_1 and M_2
3. $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$
4. $q_0 = (q_1, q_2)$
5. $F = \{(r_1, r_2) : r_1 \in F_1 \text{ or } r_2 \in F_2\}$



Theorem 4

If L_1, L_2 are regular languages, then $L_1 \cap L_2$ is also regular.

Same Cartesian product construction as union, except:

$$F = \{(r_1, r_2) \in Q_1 \times Q_2 : r_1 \in F_1 \text{ and } r_2 \in F_2\}.$$

M accepts w iff both M_1 and M_2 accept w , so
 $L(M) = L_1 \cap L_2$.

Theorem 5

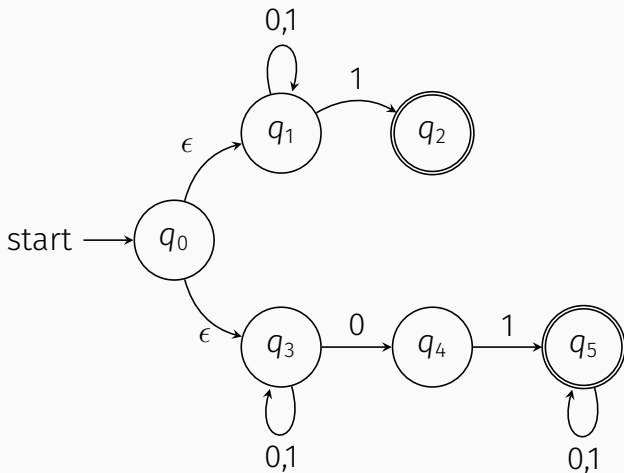
If L_1, L_2 are regular languages, then $L_1 \circ L_2$ is also regular.

We'd like to split the input into two pieces at some point, check M_1 accepts the first piece and M_2 accepts the second. But we don't know **where** to split.

⇒ Our fix: introduce a **conceptual** computational model, the **nondeterministic finite automaton (NFA)**.

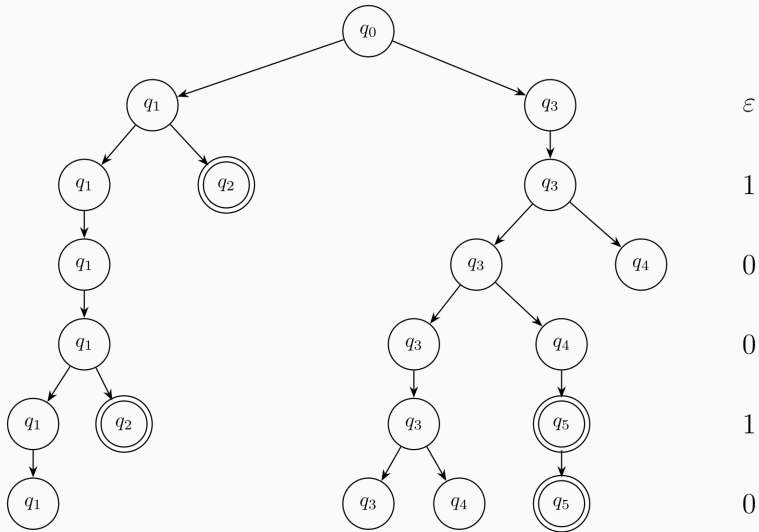
- In a DFA, the next state is **uniquely determined** by the current state and input symbol.
- In an NFA, several choices may exist for the next state — it need **not** be unique.
- An NFA accepts a string if **some** sequence of choices leads to an accepting state.

Example: NFA over $\{0, 1\}$

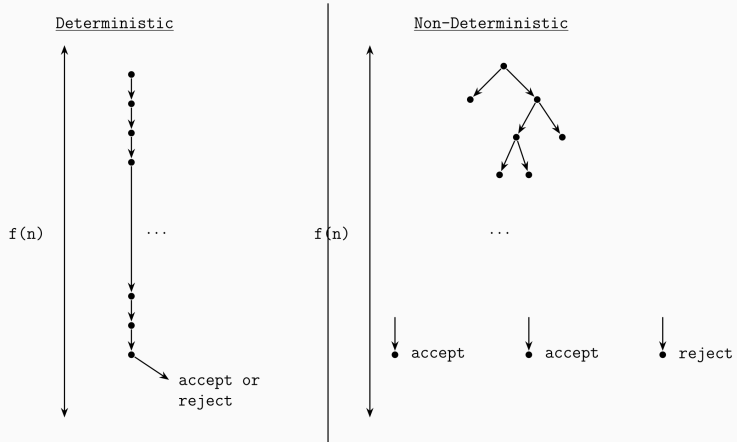


This NFA accepts strings that end with 1, or contain the substring 01.

Computation Tree: NFA on Input 10010

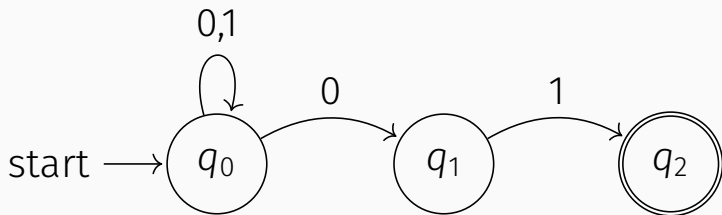


Deterministic vs. Nondeterministic Computation



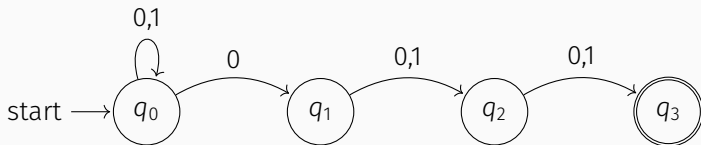
Exercise: What language does the following NFA accept?

(1).



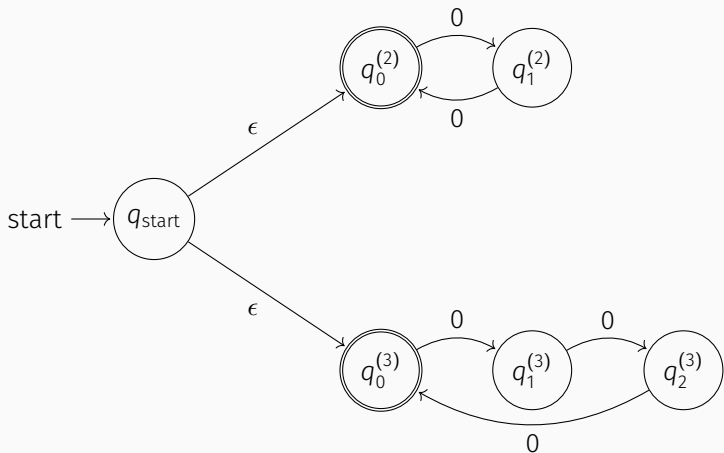
Exercise: What language does the following NFA accept?

(2).



Exercise: What language does the following NFA accept?

(3).



Definition 6

For a set Q , its *power set* $\mathcal{P}(Q)$ is the collection of all subsets of Q .

Examples:

- $Q = \{1\}$: $\mathcal{P}(Q) = \{\emptyset, \{1\}\}$
- $Q = \{1, 2\}$: $\mathcal{P}(Q) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$
- $Q = \{q_0, q_1, q_2\}$:

$$\mathcal{P}(Q) = \{\emptyset, \{q_0\}, \{q_1\}, \{q_2\}, \{q_0, q_1\}, \\ \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}$$

Definition 7 (NFA)

A *nondeterministic finite automaton* (NFA) is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set of states,
2. Σ is a finite alphabet,
3. $\delta : Q \times \Sigma_{\epsilon} \rightarrow \mathcal{P}(Q)$ is the transition function (here $\Sigma_{\epsilon} = \Sigma \cup \{\epsilon\}$),
4. $q_0 \in Q$ is the start state, and
5. $F \subseteq Q$ is the set of accept states.

Formal Definition of NFA Acceptance

Definition 8

Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA, and $w \in \Sigma^*$.

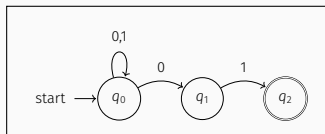
We say that N *accepts* w if we can write

$w = w_1w_2 \cdots w_n$, where each $w_i \in \Sigma_\epsilon$, and there exist $r_0, r_1, \dots, r_n \in Q$ such that

1. $r_0 = q_0$,
2. $r_i \in \delta(r_{i-1}, w_i)$, for $i = 1, \dots, n$, and
3. $r_n \in F$.

Formal Description of the NFA

Let us look at the NFA we saw earlier.



1. $Q = \{q_0, q_1, q_2\}$
2. $\Sigma = \{0, 1\}$
3. δ is given in the following table.
4. q_0 is the start state
5. $F = \{q_2\}$.

	0	1	ϵ
q_0	$\{q_0, q_1\}$	$\{q_0\}$	$\emptyset$
q_1	$\emptyset$	$\{q_2\}$	$\emptyset$
q_2	$\emptyset$	$\emptyset$	$\emptyset$

Theorem 9

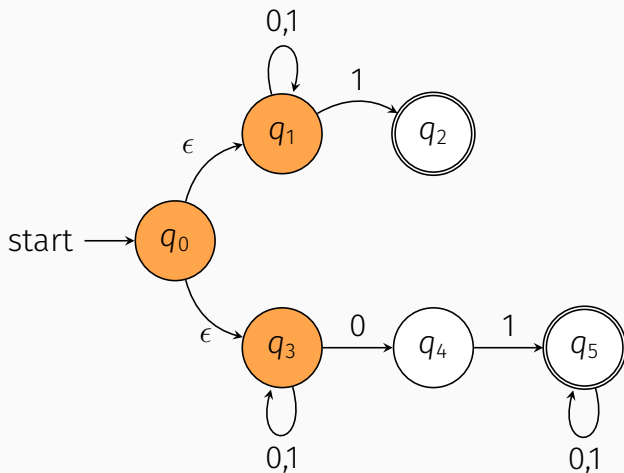
Every NFA has an equivalent DFA.

Idea: an NFA may be in several states at once, since transitions can be nondeterministic. So at each time step, we track the **set of all states** the NFA could currently be in.

This means a state of the new DFA is a **set** of NFA states — i.e., an element of $\mathcal{P}(Q)$.

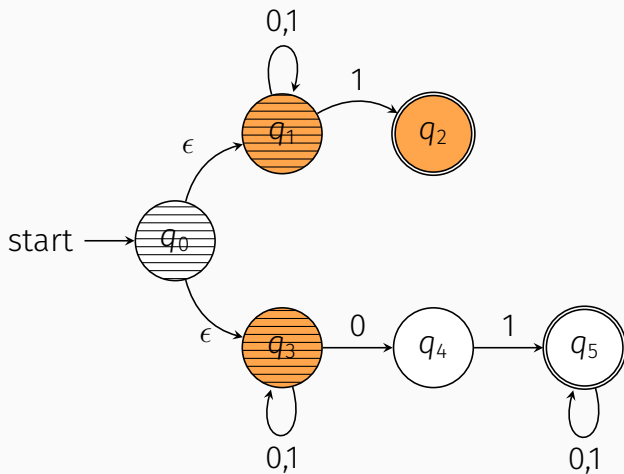
Tracking Possible States: Before Reading Anything

Input: **10010** (nothing read yet)



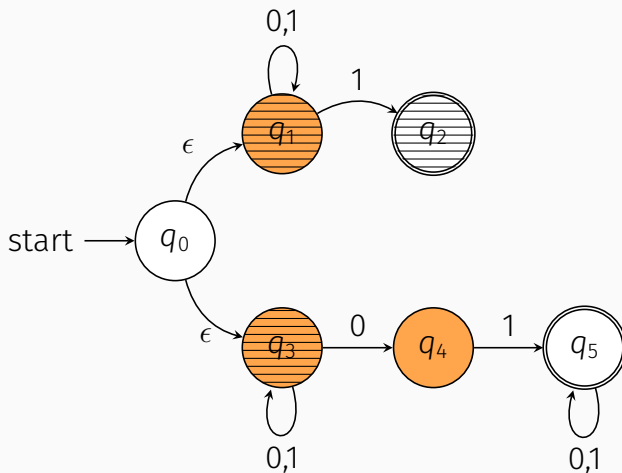
Tracking Possible States: After Reading 1

Input: **10010**



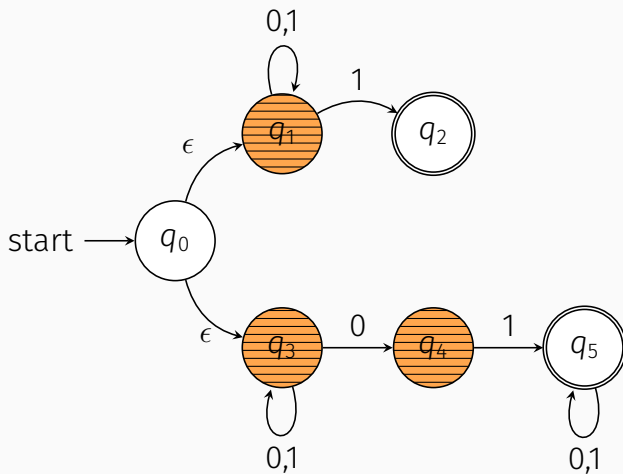
Tracking Possible States: After Reading 10

Input: **10010**



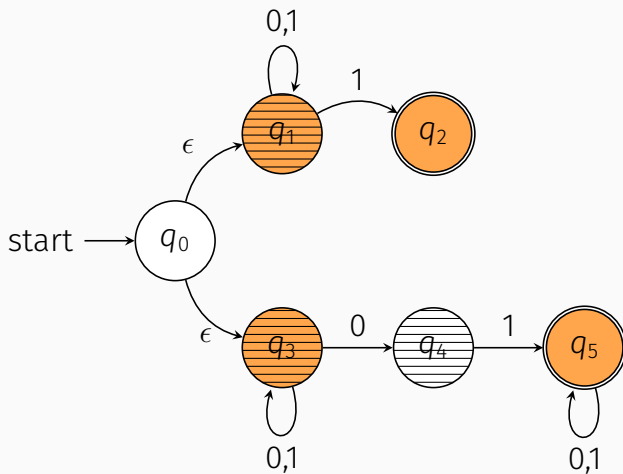
Tracking Possible States: After Reading 100

Input: **10010**



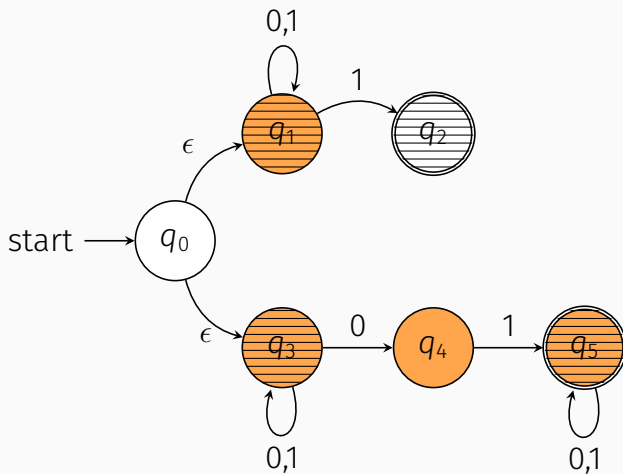
Tracking Possible States: After Reading 1001

Input: **10010**



Tracking Possible States: After Reading 10010

Input: 10010



Proof. Let $N = (Q, \Sigma, \delta, q_0, F)$ be the NFA recognizing L . Construct a DFA $M = (Q', \Sigma, \delta', q'_0, F')$ recognizing L .

First, to deal with ϵ arrows, define

$$E(R) = \{q \in Q : q \text{ can be reached from } R \text{ by} \\ \text{traveling along zero or more } \epsilon \text{ arrows}\},$$

where $R \subseteq Q$.

Construct DFA $M = (Q', \Sigma, \delta', q'_0, F')$ as follows:

1. $Q' = \mathcal{P}(Q)$.

2. For each $R \in Q'$ and $a \in \Sigma$, let

$$\begin{aligned}\delta'(R, a) &= \{q \in Q : q \in E(\delta(r, a)) \text{ for some } r \in R\} \\ &= \bigcup_{r \in R} E(\delta(r, a)).\end{aligned}$$

3. $q'_0 = E(\{q_0\})$.

4. $F' = \{R \in Q' : R \cap F \neq \emptyset\}$.

It remains to show $L(M) = L(N)$, via mutual inclusion.

Equivalence of NFAs and DFAs: $L(N) \subseteq L(M)$

$(L(N) \subseteq L(M))$ Let $w = w_1w_2 \cdots w_n$ ($w_i \in \Sigma$), for which there exists a state sequence

$$q_0, q_0^{(1)}, \dots, q_0^{(\ell_0)}, q_1, q_1^{(1)}, \dots, q_1^{(\ell_1)}, \dots, q_n, q_n^{(1)}, \dots, q_n^{(\ell_n)},$$

where $\ell_0, \dots, \ell_n \geq 0$, such that consecutive states within each block are connected by ϵ -arrows, and $q_{i-1}^{(\ell_{i-1})} \xrightarrow{w_i} q_i$ for each $i = 1, \dots, n$.

Let $R_0 = E(\{q_0\})$, and $R_i = \delta'(R_{i-1}, w_i)$ for $i = 1, \dots, n$.

Claim. $q_i, q_i^{(1)}, \dots, q_i^{(\ell_i)} \in R_i$.

We prove the claim by induction on i . The base case $i = 0$ is easy to verify.

Equivalence of NFAs and DFAs: $L(N) \subseteq L(M)$

For the inductive step, assume the claim holds for $i - 1$. It suffices to prove $q_i \in R_i$, from which $q_i^{(1)}, \dots, q_i^{(\ell_i)} \in R_i$ follows by the definition of $E(\cdot)$.

If $\ell_{i-1} = 0$, we have $q_i \in \delta(q_{i-1}, w_i)$. Since $q_{i-1} \in R_{i-1}$, we have $q_i \in R_i = \cup_{r \in R_{i-1}} E(\delta(r, w_i))$.

If $\ell_{i-1} > 0$, we have $q_i \in \delta(q_{i-1}^{(\ell_{i-1})}, w_i)$. Since $q_{i-1}^{(\ell_{i-1})} \in R_{i-1}$, we have $q_i \in R_i$.

From the claim, $R_0, R_1, \dots, R_n$ is a valid sequence of states M passes through on input $w_1 w_2 \cdots w_n$, and R_n is an accept state.

Equivalence of NFAs and DFAs: $L(N) \supseteq L(M)$

$(L(N) \supseteq L(M))$ Let $w = w_1w_2 \cdots w_n$ ($w_i \in \Sigma$) be accepted by M , and let

$$R_0 = E(\{q_0\}), \quad R_i = \delta'(R_{i-1}, w_i) \quad \text{for } i = 1, \dots, n.$$

Since $R_n \in F'$, pick $p_n \in R_n \cap F$.

Trace back. For $i = n, n-1, \dots, 1$: since

$p_i \in R_i = \bigcup_{r \in R_{i-1}} E(\delta(r, w_i))$, there exist $p_{i-1} \in R_{i-1}$ and $s_i \in \delta(p_{i-1}, w_i)$ such that p_i is reachable from s_i along ϵ -arrows.

Finally, $p_0 \in R_0 = E(\{q_0\})$, so p_0 is reachable from q_0 along ϵ -arrows.

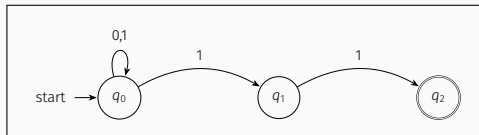
Putting these together, N has the computation

$$q_0 \xrightarrow{\epsilon^*} p_0 \xrightarrow{w_1} s_1 \xrightarrow{\epsilon^*} p_1 \xrightarrow{w_2} s_2 \xrightarrow{\epsilon^*} p_2 \cdots \xrightarrow{w_n} s_n \xrightarrow{\epsilon^*} p_n \in F,$$

where $\xrightarrow{\epsilon^*}$ denotes zero or more ϵ -arrows.

Hence N accepts w , i.e., $w \in L(N)$. □

Example: NFA to DFA – Subset Construction

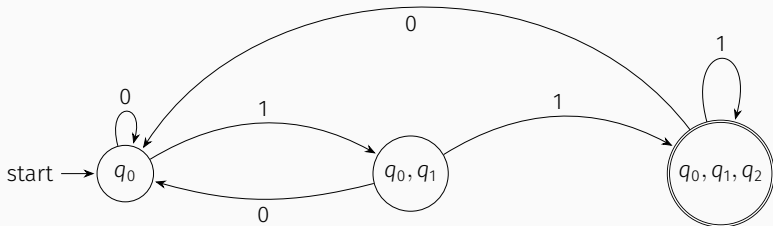


State	0	1
$\rightarrow \{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0\}$	$\{q_0, q_1, q_2\}$

Any DFA state containing q_2 becomes an accept state.

Example: NFA to DFA – DFA Construction

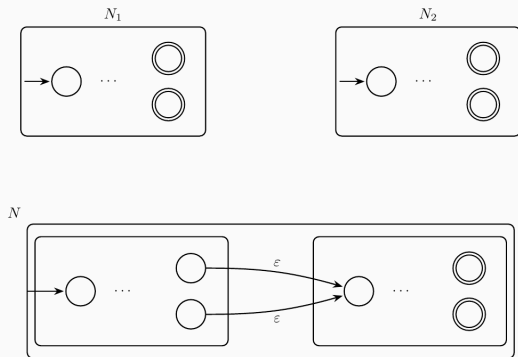
State	0	1
$\rightarrow \{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0\}$	$\{q_0, q_1, q_2\}$



Closure under Concatenation

Theorem 10

If L_1, L_2 are regular languages, then $L_1 \circ L_2$ is also regular.



Proof.

Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize language L_1 , and $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize language L_2 .

We construct $N = (Q, \Sigma, \delta, q_1, F_2)$ to recognize $L_1 \circ L_2$ as follows:

1. $Q = Q_1 \cup Q_2$
2. q_1 is the start state
3. F_2 is the set of accept states

4. For any $q \in Q$ and $a \in \Sigma_\epsilon$, define

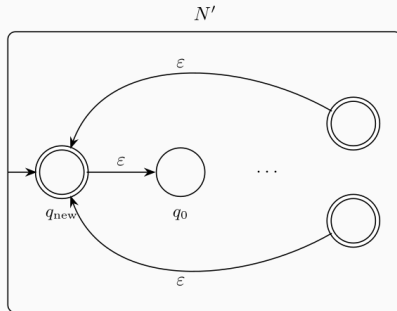
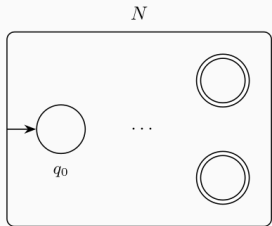
$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ and } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ and } a = \epsilon \\ \delta_2(q, a) & q \in Q_2 \end{cases}$$



Closure under Star

Theorem 11

If L is a regular language, then L^ is also regular.*



Closure under Star

Proof.

Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA that recognizes language L . We construct another NFA $N' = (Q', \Sigma, \delta', q_{\text{new}}, F')$ to recognize L^* as follows:

1. $Q' = Q \cup \{q_{\text{new}}\}$, where q_{new} is a new start state
2. $F' = F \cup \{q_{\text{new}}\}$
3. For any $q \in Q'$ and $a \in \Sigma_\epsilon$, define

$$\delta'(q, a) = \begin{cases} \delta(q, a) & q \in Q \text{ and } q \notin F \\ \delta(q, a) & q \in F \text{ and } a \neq \epsilon \\ \delta(q, a) \cup \{q_{\text{new}}\} & q \in F \text{ and } a = \epsilon \\ \{q_0\} & q = q_{\text{new}} \text{ and } a = \epsilon \end{cases}$$

Summary: Closure Properties of Regular Languages

The class of regular languages is **closed** under the following operations.

That is, if L_1, L_2 are regular languages, then so are:

- **Union:** $L_1 \cup L_2$
- **Concatenation:** $L_1 \circ L_2$
- **Star:** L_1^*
- **Complement:** $\overline{L_1}$
- **Intersection:** $L_1 \cap L_2$
- **Difference:** $L_1 - L_2$

Reference: Sipser, *Introduction to the Theory of Computation* (3rd ed.), Sections 1.1–1.2.